

Security under Paging and Segmentation

Fundamentals of Operating System Design

Prashant Jadav

DEPSTAR - CHARUSAT

Learning Objectives

By the end of this lecture, students should be able to:

- Explain how paging and segmentation enforce memory protection.
- Distinguish address translation from access-control enforcement.
- Explain read, write, execute and privilege checks.
- Analyze page-level and segment-level protection.
- Explain isolation, sharing, and controlled access.
- Analyze protection violations and fault handling.
- Compare security trade-offs of paging, segmentation, and segmented paging.
- Solve numerical and GATE-style protection questions.

Why Is Memory Protection Necessary?

Without memory protection, one process could:

- read another process's private data,
- overwrite another process's code,
- modify kernel memory,
- execute data that should not be executable,
- corrupt operating-system structures.

The OS therefore needs:

Isolation + Access Control + Controlled Sharing

Example

A web browser process should not be able to write into the kernel's page tables simply because both reside in physical memory.

Security Goals of Memory Management

Goal	Meaning
Isolation	One process cannot arbitrarily access another process's memory.
Confidentiality	Unauthorized reads are prevented.
Integrity	Unauthorized writes/modification are prevented.
Execute control	Data pages can be prevented from executing.
Privilege separation	User code is restricted from privileged memory.
Controlled sharing	Shared memory is explicitly mapped and permitted.
Fault containment	Invalid references are detected before harmful access.

Security Starts with Address Translation

A virtual address is not automatically a physical address.



The hardware/MMU checks whether:

- the address is mapped,
- the access type is permitted,
- the privilege level is permitted.

Important

A successful address translation does not necessarily mean that the access is authorized.

Security under Segmentation

A segmented address is:

$$(s, d)$$

The segment descriptor can contain:

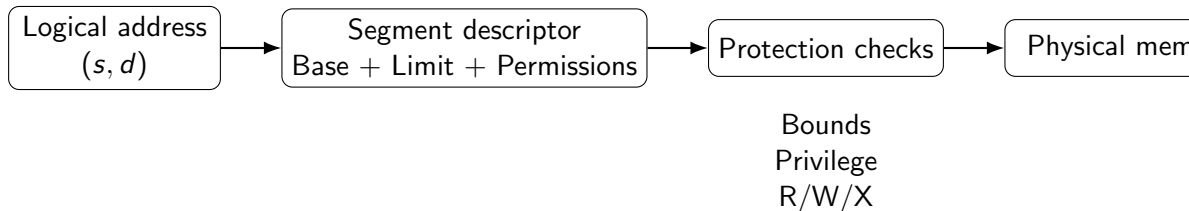
- base address,
- limit,
- read/write/execute permissions,
- privilege information,
- present/valid information.

Translation and protection:

$$d < Limit_s \quad \wedge \quad AccessType \in Permissions_s$$

Only if both conditions are satisfied is the memory reference allowed.

Segmentation Protection: Diagram



$PA = Base_s + d$ only after protection checks succeed

Segmentation: Bounds Protection

Suppose:

$$Base = 40000, \quad Limit = 5000$$

Logical address:

$$(s, 3200)$$

Since:

$$3200 < 5000$$

the reference is within the segment.

$$PA = 40000 + 3200 = \boxed{43200}$$

Now consider offset:

$$5200$$

Since:

$$5200 > 5000$$

the reference is rejected:

Segmentation: Permission Protection

Consider three segments:

Segment	Permission
Code	Read + Execute
Data	Read + Write
Stack	Read + Write

If a program attempts:

`write(Code)`

the bounds may be valid, but:

$\text{Write} \notin \text{Permissions}_{\text{Code}}$

Therefore:

Protection fault

This demonstrates the difference between **address validity** and **access authorization**.

Privilege Levels in Segmentation

A protection mechanism can distinguish privilege levels.

For example:

Kernel mode > User mode

A segment may be accessible only at an appropriate privilege level.

- User code accesses user segments.
- Kernel code can access privileged kernel segments.
- A user process cannot simply change its own privilege level to access protected memory.

Key idea

Protection information is enforced by the processor/MMU, not merely by application software.

Security under Paging

A paged address is:

$$(p, d)$$

The page-table entry can contain protection-related bits such as:

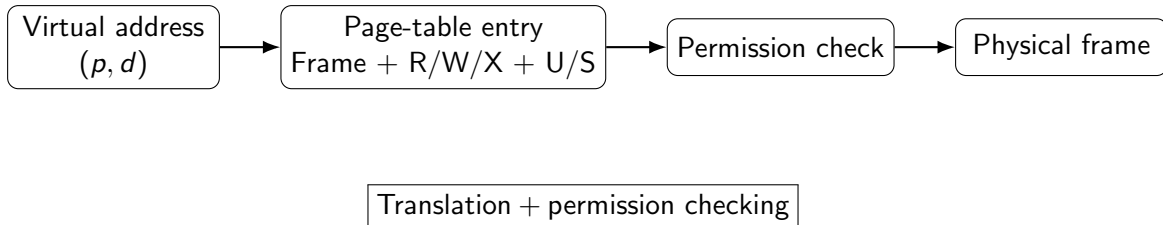
- present/valid,
- read/write,
- user/supervisor,
- execute-disable / NX,
- accessed,
- dirty.

Thus:

$$\text{Page number} \rightarrow \text{frame} + \text{permissions}$$

The page table is therefore both a translation structure and an important protection structure.

Paging Protection: Diagram



User/Supervisor Protection

A common paging protection model distinguishes:

User pages

Supervisor/kernel pages

Example:

Page	Mode	Typical use
P0	User	Application code
P1	User	Application data
P2	User	Stack
P3	Supervisor	Kernel data
P4	Supervisor	Kernel code

A user-mode access to a supervisor-only page causes a:

protection fault

Read/Write/Execute Protection

$R = \text{Read}, \quad W = \text{Write}, \quad X = \text{Execute}$

Example page permissions:

Page	Permissions
<i>Code</i>	$R - X$
<i>Data</i>	$RW -$
<i>Read - only data</i>	$R - -$
<i>Stack</i>	$RW -$

A page marked:

$RW -$

should not be executable.

Security benefit

Separating writable data from executable code reduces the consequences of certain memory-corruption attacks.

Numerical Example: Page Permission Check

Suppose:

Page 7 $\rightarrow$ *Frame 12*

Page 7 has:

Permissions = *R* – *X*

A process performs:

Read(*Page 7*)

Result:

Allowed

If it performs:

Write(*Page 7*)

then:

Protection fault

The frame mapping is valid, but the requested operation is not authorized.

Present/Valid Bit

A page-table entry may contain a present/valid bit.

$Valid = 1 \Rightarrow$ page has a valid mapping

$Valid = 0 \Rightarrow$ reference cannot be completed directly

Depending on the system, a non-present page may lead to:

- page fault,
- demand paging,
- invalid-address exception.

Important

A page fault is not automatically a security violation. It may simply mean that the page is currently not resident.

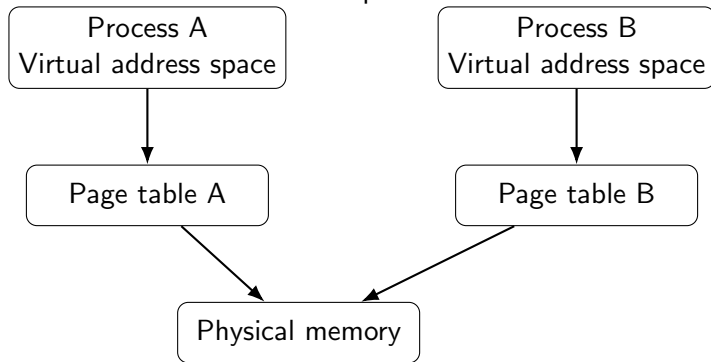
Page Fault vs. Protection Fault

	Page fault	Protection fault
Page mapping	May be valid but not resident	Access may be unauthorized
Typical cause	Not-present page	R/W/X or privilege violation
OS action	May load page	Usually reject/terminate or signal
Security meaning	Not necessarily an attack	Indicates a policy violation

Not present $\neq$ not permitted

Isolation Between Processes

Each process can have its own virtual address space.



Different virtual addresses can map to different physical frames even when the numeric virtual addresses are identical.

Isolation: Numerical Example

Suppose:

Process A : VA 1000 $\rightarrow$ Frame 5

Process B : VA 1000 $\rightarrow$ Frame 12

Both processes use:

$$VA = 1000$$

But:

$$PA_A = 5 \times PageSize + Offset$$

$$PA_B = 12 \times PageSize + Offset$$

Therefore:

Same virtual address does not imply same physical memory

This is fundamental to process isolation.

Controlled Sharing under Paging

Suppose two processes need a shared read-only library.

Process A page $\longrightarrow$ Physical frame F $\longleftarrow$ Process B page

Both page tables map their virtual pages to the same physical frame.

Permissions:

$R - X$

Thus:

Sharing without giving write access

This provides both efficiency and controlled access.

Controlled Sharing under Segmentation

Segmentation can represent a shared logical object directly.

Example:

Shared code segment

Two processes may have:

Segment 2 → same physical region

with:

$$\text{Permissions} = R - X$$

This naturally expresses:

shared logical unit + common protection policy

Paging vs. Segmentation: Security Comparison

Security property	Paging	Segmentation
Isolation	Page-table mappings	Segment boundaries + mappings
Bounds checking	Page-level mapping	Explicit segment limit
R/W/X control	Per page	Per segment, implementation-dependent
Privilege	Page permissions	Segment privilege checks
Sharing	Page granularity	Logical segment granularity
Logical protection domains	Less direct	Natural
Fine-grained memory control	Strong	Segment-size dependent
Virtual memory integration	Strong	More complex alone

Segmentation with Paging: Security

Logical address:

$$(s, p, d)$$

Security can be enforced at multiple levels:

Segment check $\rightarrow$ Page check $\rightarrow$ Physical access

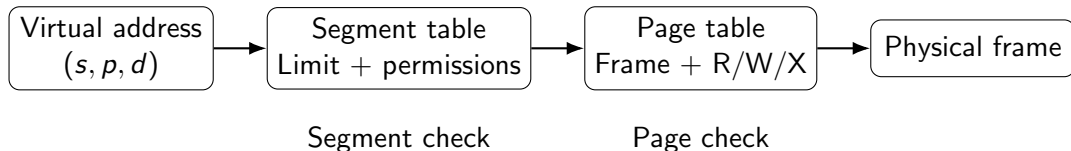
Segment level:

- logical boundary,
- privilege,
- segment permissions.

Page level:

- present bit,
- R/W/X,
- user/supervisor,
- frame mapping.

Security in Segmented Paging: Diagram



Two levels of organization can provide two levels of protection

Security vs. Performance Trade-off

More protection checks can introduce:

- additional metadata,
- additional translation work,
- TLB/cache pressure,
- more complex hardware.

However, protection mechanisms are often implemented in hardware and cached.

Security enforcement	$\longleftrightarrow$	translation/performance overhead
----------------------	-----------------------	----------------------------------

Design principle

Do not remove protection merely to reduce a small translation overhead; evaluate the security requirement and the actual measured cost.

Numerical Example: Protection-Check Cost

Assume:

$$T_M = 100 \text{ ns}$$

and a simplified design requires:

1 memory access for translation + 1 memory access for data

Then:

$$EAT = 200 \text{ ns}$$

If a TLB hit supplies the frame and permissions:

$$EAT \approx T_{TLB} + T_M$$

For:

$$T_{TLB} = 10 \text{ ns}$$

$$EAT \approx 10 + 100 = \boxed{110 \text{ ns}}$$

The TLB improves performance while retaining cached translation/protection information.

Security Example: Write to Read-Only Code

Page 4 $\rightarrow$ Frame 20 and Permissions(Page 4) = R - X

and Instruction:

Store VA

Translation succeeds:

Page 4 $\rightarrow$ Frame 20

But:

Write $\notin$ {R, X}

Therefore:

Protection violation

Important

A valid page mapping does not imply that every operation on the page is valid.

Security Example: User Access to Kernel Page

Suppose a page-table entry contains:

$$U/S = \textit{Supervisor}$$

Current CPU mode:

User mode

User process attempts:

Read(Page)

Since:

User mode $\neq$ required supervisor privilege

the processor rejects the access:

Protection fault

This prevents ordinary applications from directly reading protected kernel memory.

Security Example: Out-of-Bounds Segment Access

Segment:

$$Base = 10000, \quad Limit = 3000$$

Request:

$$Offset = 2500$$

Valid:

$$2500 < 3000$$

$$PA = 10000 + 2500 = \boxed{12500}$$

Now:

$$Offset = 3500$$

$$3500 > 3000$$

Therefore:

Access rejected

Security Properties and Attack Surface

Memory protection helps limit the impact of:

- buffer overflows,
- invalid pointer dereferences,
- accidental memory corruption,
- attempts to modify read-only code,
- attempts to access privileged memory.

Protection mechanisms do **not** by themselves guarantee:

- bug-free software,
- complete system security,
- protection against every side-channel attack.

Important

Paging/segmentation protection is one layer of the OS security architecture, not the entire security model.

TLB and Security

A TLB caches address translations.

A TLB entry may effectively cache information associated with:

Virtual page $\rightarrow$ physical frame + permissions

Security requires:

- correct association with the current address space,
- correct privilege information,
- proper invalidation/context-switch handling.

Security risk if context information is incorrect

A stale translation could potentially associate one process's virtual address with another process's physical mapping.

ASID/PCID: Reducing TLB Context-Switch Cost

Modern processors may associate TLB entries with an address-space identifier.
Conceptually:

$$(ASID, VPN) \rightarrow \text{Frame} + \text{Permissions}$$

This can reduce unnecessary TLB flushing during context switches.
Benefits:

- fewer TLB misses,
- lower context-switch overhead,
- safer separation of address spaces when tags are handled correctly.

Performance optimization must preserve address-space isolation

Security Comparison: Fixed vs. Variable Granularity

Paging:

Protection granularity = page

Segmentation:

Protection granularity = segment

Example:

8 KB logical data object

If page size = 4 KB:

2 pages

The object may need multiple page permissions.

A single 8-KB segment can naturally represent the entire logical object.

Trade-off

Fine granularity gives precise control but increases the number of protection entries.

GATE-Style Question 1: Segmentation

A segment has:

$$Base = 12000, \quad Limit = 4000$$

Determine whether:

$$(2, 3500)$$

is a valid reference, and if valid calculate the physical address.

$$3500 < 4000$$

Therefore:

Valid

$$PA = 12000 + 3500 = 15500$$

If the offset were 4500:

Protection fault

GATE-Style Question 2: Paging Permissions

A page table contains:

Page 6 $\rightarrow$ *Frame 15*

with permissions:

$R - X$

Classify:

- 1 instruction fetch,
- 2 read,
- 3 write.

Answer:

Fetch: Allowed

Read: Allowed

Write: Protection fault

The page mapping itself is valid; only the requested operation is unauthorized.

GATE-Style Question 3: TLB + Protection

Suppose:

$$T_{TLB} = 10 \text{ ns}$$

$$T_M = 100 \text{ ns}$$

$$h = 0.95$$

Assume the TLB stores both:

frame number + access permissions

Then:

$$\begin{aligned} EAT &= 0.95(10 + 100) + 0.05(10 + 200) \\ &= 104.5 + 10.5 = \boxed{115 \text{ ns}} \end{aligned}$$

Concept

The cached translation can also provide the permission information needed for the access check.

GATE-Style HOT Question

A student says:

"If a virtual address is successfully translated to a physical address, the access must be legal."

Is the statement correct?

Analysis:

- Translation identifies a physical location.
- Protection bits determine whether the requested operation is allowed.
- User/supervisor state can further restrict access.
- R/W/X permissions may reject an otherwise valid mapping.

Translation success $\neq$ authorization success

Common Examination Mistakes

- Confusing a page fault with a protection fault.
- Assuming every valid page is writable.
- Forgetting user/supervisor privilege checks.
- Treating segmentation limit as a physical-address limit.
- Assuming paging provides security automatically without page permissions.
- Assuming segmentation automatically prevents every form of memory attack.
- Forgetting that shared pages must have intentionally compatible permissions.
- Ignoring TLB/address-space context when discussing isolation.

Security Design Comparison

Property	Paging	Segmentation
Isolation	Per-page mappings	Segment boundaries/mappings
Bounds protection	Page mapping + address validity	Base/limit check
R/W/X	Per-page	Per-segment in supporting designs
Privilege separation	User/supervisor page bits	Segment privilege checks
Sharing	Page granularity	Logical segment granularity
Fine-grained control	Strong	Depends on segment size
Virtual memory	Strong integration	More complex alone
Metadata	Page tables	Segment descriptors
Main security strength	Fine-grained mapping/protection	Logical protection domains

Design Trade-offs

Paging

- Fine-grained page protection.
- R/W/X and privilege bits.
- Strong process isolation.
- Natural virtual-memory integration.

Segmentation

- Logical protection domains.
- Base/limit checking.
- Natural logical sharing.
- Protection follows program structure.

Segmented Paging

- Logical segment boundaries.
- Page-level protection.
- Multiple metadata structures.
- Higher translation complexity.

Design principle

Security policy should match the required protection granularity and the workload.

Real-World Analogy: Building Security

Segmentation analogy:

- Building divided into rooms/departments.
- Each room has a purpose and access policy.

Paging analogy:

- Building divided into standardized access zones.
- Each zone can have independent access permissions.

Combined:

Logical rooms + fine-grained zones

The analogy illustrates protection granularity; actual OS enforcement is performed by hardware/MMU mechanisms.

HOT Design Scenario

A system runs:

- a kernel,
- multiple user applications,
- shared read-only libraries,
- writable application heaps,
- executable code.

Design a protection policy using paging.

Expected design:

Region	Suggested permissions
Code	R-X
Read-only data	R-
Heap	RW-
Stack	RW-
Shared library code	R-X
Kernel memory	Supervisor only

HOT Analytical Question: Compare two designs

Design A:

Paging with per-page R/W/X + user/supervisor bits

Design B:

Segmentation with logical protection domains

Analyze:

- 1 isolation,
- 2 protection granularity,
- 3 sharing,
- 4 metadata overhead,
- 5 address translation,
- 6 compatibility with virtual memory,
- 7 ease of expressing logical protection domains.

Expected answer

Justify the design according to workload and security requirements.

Key Formula and Rule Sheet

Segmentation:

$$PA = Base_s + Offset, \quad Offset < Limit_s$$

Paging:

$$PA = Frame_p \times PageSize + Offset$$

Access authorization:

$$ValidMapping \wedge PermissionOK \wedge PrivilegeOK$$

TLB EAT:

$$EAT = h(T_T + T_M) + (1 - h)(T_T + 2T_M)$$

Core security rule:

$$\text{Valid translation} \neq \text{valid access}$$

Summary

- Memory protection provides isolation, confidentiality, integrity and controlled sharing.
- Segmentation protects logical regions using base, limit and permission information.
- Paging protects individual pages using page-table permission and privilege bits.
- R/W/X permissions can separate executable code from writable data.
- User/supervisor controls restrict access to privileged memory.
- Page faults and protection faults have different meanings.
- TLBs improve translation performance while also caching protection-related information.
- Segmentation with paging can combine logical protection domains with page-level protection.

Key takeaway

Address translation answers “where is the memory?”; protection mechanisms additionally answer “is this access allowed?”